

基本原则	Principles
清晰具体	写明目标、动作、对象,避免"帮我看"
给上下文	背景、约束、已知信息一次到位
设角色	"你是资深 X 工程师" 校准语气与深度
说受众	面向新手 / 专家 / CEO,措辞不同
定格式	指定 markdown / JSON / 表格 / 字段
加约束	长度、语言、范围、禁用词
拆步骤	复杂任务分阶段,逐步交付
例子优于解释	2-3 个 few-shot 比一段说明更准
先思考再答	<code><thinking></code> / "step by step" 给推理空间
正面指令	"做 Y" 比 "不要做 X" 有效
定义术语	领域词先解释,避免歧义
给评判标准	告诉模型"好答案"长什么样
心法	把 LLM 当 聪明但失忆的实习生:背景越足、目标越明,产出越稳

高级技巧	Advanced
Meta-prompting	让 AI 改 / 评分 / 重写你的 prompt
Prompt Chaining	分解为多步,前步输出喂下一步
Recursive Critique	draft → critique → revise 循环
Self-refine	同模型自我改进 N 轮
RAG-augmented	检索片段塞进 context 再问
Function calling	强制 schema,稳得多
Constrained decoding	grammar / regex 限制输出
Ensemble	多模型 / 多 prompt 投票或合并
Router prompt	小模型分类 → 路由到大模型
Reflexion	失败后写"反思"再重试
Skeleton + fan-out	先大纲并行展开,加速
提示注入防御	用户输入用 XML/分隔符隔离 + system 反复声明边界 + 输出层做后置校验

提示模式	Patterns
Zero-shot	直接给指令,不带例子
Few-shot	2-5 个 input/output 示范,覆盖边界
Chain-of-Thought	"Let's think step by step" 显式推理
Zero-shot CoT	无例子 + 思考触发短语
Self-consistency	多次采样取多数,数学/逻辑更稳
ReAct	Reason + Act 循环,配工具调用
Tree-of-Thoughts	分支探索 + 评分剪枝
Self-Ask	自问子问题再合并答案
Generated Knowledge	先让模型列相关知识再答题
Step-back	先问抽象/原理问题,再回到具体
Plan-and-Solve	先出计划,再按计划执行
Skeleton-of-Thought	先列大纲再并行展开各点
速记	简单 → Zero-shot,有边界 → Few-shot,需推理 → CoT/ToT,需工具 → ReAct

平台差异	Platforms
Claude (Anthropic)	
XML 标签优先	<code><context></code> <code><example></code> 比 markdown 稳
prompt caching	长 system 命中 90% 折扣
extended thinking	opus/sonnet 可开 thinking 预算
prefilling 强	assistant 开头可控
GPT (OpenAI)	
markdown + system	标题列表组织清晰
JSON mode	<code>response_format</code> 保证合法
Structured Outputs	100% schema 一致
developer role	优先级高于 user
Gemini (Google)	
system_instruction	独立字段
2M context	超长文档/代码库友好
多模态原生	video/audio/PDF 直接喂
response_schema	JSON 严格输出

结构化输出	Structured Output
<code><context></code>	Anthropic 偏好 XML 标签包裹长文本
<code><instructions></code>	指令独立成段,易于切换
<code><example></code>	few-shot 用标签清晰分隔
<code><thinking></code>	让模型先在标签内推理再输出
JSON mode	OpenAI <code>response_format=json_object</code>
JSON Schema	OpenAI Structured Outputs 严格校验
tool_choice	强制走 function 拿到 schema 输出
prefilling "{"	Anthropic 在 assistant 起始填 <code>{</code> 锁 JSON
response_schema	Gemini 显式 schema + MIME
stop sequences	在 <code>}</code> 或 <code></answer></code> 处止住
提示骨架	"Output ONLY valid JSON matching this schema: <code>{...}</code> . No prose."
XML vs Markdown	Claude 用 XML 更稳;GPT/Gemini markdown 标题也行,但 schema 校验首选

Prompt Caching	Cache 折扣
Anthropic <code>cache_control</code>	
<pre>client.messages.create(model="claude-opus-4-7", system=[{"type":"text", "text":"<long_doc>...</long_doc>", "cache_control":{"type":"ephemeral"}}], messages=[{"role":"user","content":"问题"}])</pre> # TTL: 5min(默认) / 1h # 写入 1.25× · 命中 0.1×(90% off) # 最多 4 个 breakpoint,前缀共享	
OpenAI	自动缓存 ≥1024 tokens 前缀,无需配置;命中约 50% 折扣 · 头部稳定即可
Gemini	显式 <code>cachedContent</code> + <code>ttl</code> ;先 <code>caches.create()</code> 再引用 ID,适合长文档复用
命中要点	
前缀稳定	把可变部分放最后
顺序固定	tools / system / docs 不要换序
测命中率	看 <code>cache_read_input_tokens</code>

消息角色	Roles
system	身份 + 约束 + 风格 + 边界
user	本轮任务和数据
assistant	历史回复或 prefix 起始
tool / function	工具返回结果(角色独立)
system 写法	"你是 X · 必须 Y · 禁止 Z · 输出格式 W" — 短而锐,别堆故事
prefilling(Anthropic)	在 messages 末尾加 <code>{"role":"assistant","content":""}</code>
强制 JSON 起始 / 控制语气	
multi-turn 设计	
固定 system	放可缓存的长指令
滚动 user	最新输入放最后
历史摘要	超长对话先摘要再继续
显式 reset	"忽略以上,从头开始" 切换任务
注意	Gemini 用 <code>system_instruction</code> 字段;OpenAI 也支持 <code>developer</code> 角色(优先级 > user)

Pro Tips & 反模式	Pro Tips
▶ 不要写 "不要做 X",改成 "做 Y" — 否定句模型常忽略	
▶ 给 AI 思考空间: <code><thinking>...</thinking></code> 或 "step by step"	
▶ 例子要覆盖 边界 / 异常 / 否定情况,只给正例会过拟合	
▶ 温度:确定性任务 0-0.3,创意 0.7-1.0;JSON 用低温	
▶ system 别写小作文,300-800 字最甜;长文丢 cache 段	
▶ 用 eval 集 测 prompt(20-50 例),改一句跑一遍	
▶ 别相信 LLM "我以前做过 / 我记得",每次都要喂数据	
▶ 变量用 <code>{{name}}</code> 模板化,别手拼字符串	
▶ 长文档放 system 顶部,问题放 user 末尾,缓存友好	
▶ JSON 失败兜底:prefix <code>{</code> + stop <code>}</code> + try/except 修复	
▶ "角色扮演 + 任务" 比纯任务效果好,但别堆叠 5 个角色	
▶ 遇到拒答:换措辞 / 拆任务 / 给合法用途上下文	
▶ 幻觉防御:要求 "不知道就说不知道" + 引用源	
▶ 少用 "请 / 麻烦",礼貌词浪费 token,效果不变	
▶ 多语言任务:在 system 钉死输出语言,user 切换不影响	
▶ prompt 也要做 版本化 + diff,像代码一样 review	